

▼ Introduction

This notebook prepares a dataset of human, avatar, and animal faces for a machine learning image classification task. It downloads data from Google Drive and Kaggle, applies filtering based on age (for humans) and image quality (min dimension, aspect ratio). Human data is sampled using stratification to maintain demographic balance. All images are deduplicated based on visual content, center-cropped to a square, and resized to a fixed dimension. The processed images are organized into an ImageFolder structure with train, validation, and test splits, and a CSV is generated mapping human images to their labels for traceability.

Target Dataset Split Sizes:

Split	Total Images	Human	Avatar	Animal
Train	24000	8000	8000	8000
Validation	3000	1000	1000	1000
Test	3000	1000	1000	1000

Note that the actual number of images in the final ImageFolder might be slightly less than these target numbers, particularly for the human class, due to image quality filters applied during the processing step.

Techniques

Filtering

- Age Filtering (for humans): In the `load_fairface_df` function, the FairFace data is filtered to include only specific "working age" groups ('20-29' to '60-69'), excluding individuals under 20 or over 69 based on their provided age labels.
- Image Quality Filtering: Throughout the pipeline, particularly during data loading and processing, a `check_image_rules` function is applied. This filters out images that don't meet minimum dimension requirements or fall outside acceptable aspect ratio ranges.

Image Rules

- Readability: It first attempts to open and read the image file. If it cannot be read, it fails the check.
- Minimum Dimension: If `APPLY_IMAGE_FILTERS` is True, it checks if the shorter side of the image (minimum of width and height) is less than a specified minimum pixel value (`MIN_SHORT_SIDE_PX`). Images smaller than this minimum fail the check.
- Aspect Ratio: If `APPLY_IMAGE_FILTERS` is True, it calculates the image's aspect ratio (width divided by height) and checks if it falls outside a specified acceptable range (`MIN_ASPECT_RATIO` to `MAX_ASPECT_RATIO`). Images with aspect ratios outside this range fail the check.

Deduplication

Deduplication in this notebook identifies and removes duplicate images based on their visual content, not just their filenames or metadata. It uses a technique called perceptual hashing (pHash) to create a unique "fingerprint" for each image. If two images have the same perceptual hash, they are considered duplicates, and only one is kept. This ensures that the dataset used for training or evaluation contains distinct images, preventing the model from being overly exposed to identical or near-identical examples.

Stratification

Image stratification is applied to the human data using the `sample_n_stratified` function based on combined demographic "strata" (race, gender, and age). We do this to ensure the sampled subset of human images maintains the same proportions of these demographic groups as the larger filtered FairFace dataset. This helps prevent bias in the dataset by ensuring representation across different demographics, which is crucial for training fair and robust models.

Cropping and Resizing

all images are processed using the `center_crop_resize` function during the `copy_and_process` step. This involves first centering and cropping each image to a square aspect ratio. Then, the square image is resized to a fixed dimension, specified by `IMG_SIZE`. This process standardizes the size and aspect ratio of all images in the dataset, which is a necessary preprocessing step for many machine learning models.

Dataset Builder

- OUTPUT: `data/final/{train,val,test}/{human,avatar,animal}/*.jpg` (224x224 JPEG)
- TARGET: ResNet50 fine-tuning (PyTorch/TF compatible ImageFolder)
- TARGET: Human Labels (CSV) - Metadata for human images in dataset.

Datasets



- [Human faces \(FairFace repo\)](#)
 - [Images \(padding=1.25\)](#)
 - [Labels \(train\)](#)
 - [Labels \(val\)](#)
- [Avatars \(Google's Cartoon Set via Kaggle\)](#)
- [Animal faces \(Dogs vs Cats via Kaggle\)](#)

Important Notes

- GitHub seems to have trouble rendering this notebook when output is included. The checked in version has no output because of that. I'm working on finding a solution for this issue.
- The 'salader/dogs-vs-cats' dataset has been removed from Kaggle.

Summary:

Data Analysis Key Findings

- The processed data for human, avatar, and animal classes were successfully collected and stored as lists of dictionaries.
- These individual lists of dictionaries were combined into a single list named `all_processed_data`.
- The combined list was converted into a pandas DataFrame named `dataset_manifest_df`.
- The columns in `dataset_manifest_df` were renamed from 'new_path' to 'path' and 'class' to 'label'.
- The `dataset_manifest_df` was filtered to include only the 'split', 'label', and 'path' columns.
- The resulting `dataset_manifest_df` was successfully saved as a CSV file named "dataset_manifest.csv" in the specified output directory (`OUT_DIR`).
- It was determined that the previously generated CSVs, `human_labels.csv` and `fairface_combined_labels.csv`, should be kept because they contain valuable information (specific human demographic labels and original FairFace data manifest) not present in the final `dataset_manifest.csv`, useful for traceability and further analysis.

```
!pip -q install gdown imagehash kaggle --upgrade
```

```
import time
start_time = time.time()

import os, io, sys, json, math, shutil, random, zipfile, tarfile, hashlib, gdown, imagehash
from pathlib import Path
from collections import defaultdict, Counter

import numpy as np
import pandas as pd
from PIL import Image, ImageOps
from tqdm.auto import tqdm
if 'google.colab' in sys.modules:
    from google.colab import userdata

# Do I need these?
# from google.colab import drive
# import shutil
from pathlib import Path
RUNNING_IN_COLAB = True
else:
    RUNNING_IN_COLAB = False
```

Reproducibility

```
SEED = 42
random.seed(SEED);
np.random.seed(SEED)
```

USER TUNABLES (easy knobs)

```

TARGET_PER_CLASS = 10_000 # (#3) change this to scale dataset size per class
SPLIT = {"train": 0.80, "val": 0.10, "test": 0.10}
IMG_SIZE = 224
JPEG_QUALITY = 95

# Switchable image-quality gates (your #3 request)
APPLY_IMAGE_FILTERS = True          # turn filters on/off globally
MIN_SHORT_SIDE_PX = 96              # reject if min(width, height) < this
MIN_ASPECT_RATIO = 0.50             # reject if (w/h) < this
MAX_ASPECT_RATIO = 2.00             # reject if (w/h) > this
PRINT_REJECTION_MESSAGES = True     # log every rejected image (can be verbose)

# Kaggle slugs (per your links)
KAGGLE_AVATARS_DATASET = "brendanartley/cartoon-faces-googles-cartoon-set"
KAGGLE_ANIMALFACES_DATASET = "salader/dogs-vs-cats"

# Optional: restrict to specific subfolders inside those Kaggle zips (usually not needed)
AVATAR_INCLUDE_DIRS = None
ANIMAL_INCLUDE_DIRS = None

# Safety checks
MIN_REQUIRED_PER_CLASS = TARGET_PER_CLASS

# Direct Google Drive file IDs for FairFace (padding=1.25) + labels (from joojs/fairface README)
FAIRFACE_IMG_PAD125_ID = "1g7qNOZz9wC7OfOhcPqH1EZ5bk1UFGmll"
FAIRFACE_LABEL_TRAIN_ID = "1i1L3Yqwaio7YSOCj7ftgk8ZZchPG7dmH"
FAIRFACE_LABEL_VAL_ID = "1w0dja-ezstMEp81tX1a-EYkFebev4h7D"

```

▼ Path management

```

BASE_DIR = Path.cwd()
WORK_DIR = BASE_DIR # / "data"
ZIP_DIR = WORK_DIR / "zip"
OUT_DIR = WORK_DIR / "final"
RAW_DIR = WORK_DIR / "raw"
# ZIP_DIR = RAW_DIR / "fairface"
FAIRFACE_ZIP_DIR = ZIP_DIR / "fairface"
FAIRFACE_RAW_DIR = RAW_DIR / "fairface"
for p in [ZIP_DIR, RAW_DIR, OUT_DIR, FAIRFACE_ZIP_DIR, FAIRFACE_RAW_DIR]:
    p.mkdir(parents=True, exist_ok=True)

```

▼ Dependencies / environment

```

# Do I need these?
# Does Colab already put these into env vars?

if RUNNING_IN_COLAB:
    # Set Kaggle credentials as environment variables
    os.environ['KAGGLE_USERNAME'] = userdata.get('KAGGLE_USERNAME')
    os.environ['KAGGLE_KEY'] = userdata.get('KAGGLE_KEY')
    # Define the paths (assuming BASE_DIR is defined elsewhere, e.g., Path.cwd())
    # BASE_DIR = Path.cwd()
    # COLAB_DATA_DIR = BASE_DIR / "data"
    # DRIVE_DATA_PARENT_DIR = Path("/content/drive/My Drive/Colab Data") # Change if you want a different parent dir in Drive
    # DRIVE_DATA_DIR = DRIVE_DATA_PARENT_DIR / "Capstone" # Change "your_dataset_name"
else:
    # Kaggle creds sanity
    kaggle_creds = Path.home() / ".kaggle" / "kaggle.json"
    if not kaggle_creds.exists():
        print("Kaggle credentials not found at ~/.kaggle/kaggle.json - Kaggle downloads will fail until you add them.")

```

▼ Helper utils

▼ Idempotent download helpers

```

def _any_images_in_dir(root: Path) -> bool:
    # print(f'root: {root}')
    exts = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
    return any(s.suffix.lower() in exts for s in root.glob("*") if s.is_file())

```

```

return any(p.suffix.lower() in exts for p in root.rglob('*') if p.is_file())

def _any_archives_in_dir(root: Path) -> bool:
    return any(root.rglob("*.zip")) or any(root.rglob("*.tar")) or any(root.rglob("*.tar.gz")) or any(root.rglob("*.tgz"))

def extract_all_archives(root: Path, dest: Path):
    """Extract all archives (recursively finds new ones after extraction)."""
    did_any = True
    extracted_archives = set()
    while did_any:
        did_any = False
        archives_to_extract = []

        # Only look for zip files in the root directory
        zip_files = list(root.glob("*.zip"))
        # Only look for tar files in the root directory
        tar_files = list(root.glob("*.tar")) + list(root.glob("*.tar.gz")) + list(root.glob("*.tgz"))

        archives_to_extract.extend(zip_files)
        archives_to_extract.extend(tar_files)
        # print(f'root: {root}')
        # print(f'zip_files')
        # print(f'archives_to_extract: {archives_to_extract}')

        for archive_path in archives_to_extract:
            if archive_path not in extracted_archives:
                try:
                    extract_archive(archive_path, dest)
                    did_any = True
                    extracted_archives.add(archive_path)
                except Exception as e:
                    print(f"Failed to extract {archive_path.name}: {e}")

    return

def is_fairface_present(ff_dir: Path) -> bool:
    # Look for train/val with images
    train_dirs = [
        ff_dir / "train",
        ff_dir / "val",
        ff_dir / "fairface-img-margin125-trainval" / "train",
        ff_dir / "fairface-img-margin125-trainval" / "val",
    ]
    has_imgs = any(_any_images_in_dir(d) for d in train_dirs if d.exists())
    labels_ok = (ff_dir / "fairface_label_train.csv").exists() and (ff_dir / "fairface_label_val.csv").exists()
    return has_imgs and labels_ok

def ensure_fairface_downloaded():
    """
    Ensure FairFace (padding=1.25) images and labels exist & are extracted.
    Skips re-download if already present; prints informative status.
    """
    ff_zip = FAIRFACE_ZIP_DIR / "fairface-img-margin125-trainval.zip"
    lbl_train = FAIRFACE_ZIP_DIR / "fairface_label_train.csv"
    lbl_val = FAIRFACE_ZIP_DIR / "fairface_label_val.csv"

    if is_fairface_present(RAW_DIR):
        print(f"Found existing FairFace (padding=1.25) in {RAW_DIR}; skipping download.")
        return

    print("FairFace not fully present; checking files...")
    # Download missing pieces
    if not ff_zip.exists():
        print("Downloading FairFace (padding=1.25) image pack...")
        gdown.download(id=FAIRFACE_IMG_PAD125_ID, output=str(ff_zip), quiet=True)
    else:
        print("Found FairFace image zip; skipping re-download.")

    if not lbl_train.exists():
        print("Downloading FairFace label train CSV...")
        gdown.download(id=FAIRFACE_LABEL_TRAIN_ID, output=str(lbl_train), quiet=True)
    else:
        print("Found FairFace label train CSV; skipping re-download.")

    if not lbl_val.exists():
        print("Downloading FairFace label val CSV...")
        gdown.download(id=FAIRFACE_LABEL_VAL_ID, output=str(lbl_val), quiet=True)
    else:

```

```

        print("Found FairFace label val CSV; skipping re-download.")

# Copy CSVs to FAIRFACE_DIR
print(f"Copying CSVs from {FAIRFACE_ZIP_DIR} to {FAIRFACE_RAW_DIR}...")
for csv_file in [lbl_train, lbl_val]:
    if csv_file.exists():
        shutil.copy(csv_file, FAIRFACE_RAW_DIR)
        print(f"Copied {csv_file.name}")
    else:
        print(f"Warning: {csv_file.name} not found in {ZIP_DIR}. Skipping copy.")

# Extract if images not yet extracted
if not is_fairface_present(FAIRFACE_ZIP_DIR):
    print("Extracting FairFace archives...")
    try:
        extract_archive(ff_zip, FAIRFACE_RAW_DIR)
    except Exception as e:
        print(f"Extraction issue for FairFace: {e}")
    # After extraction, we consider it "present" if images+labels are available
    if is_fairface_present(FAIRFACE_RAW_DIR):
        print("FairFace ready.")
    else:
        print("FairFace may still be incomplete; verify files under:", FAIRFACE_RAW_DIR)

def ensure_kaggle_downloaded(dataset_slug: str, source: Path, dest: Path, dataset_name_for_print: str):
    """
    Ensure Kaggle dataset is present with extracted images.
    Skips re-download if images already exist; otherwise downloads and extracts.
    """
    dest.mkdir(parents=True, exist_ok=True)

    if _any_images_in_dir(dest):
        print(f"Found existing {dataset_name_for_print} images in {dest}; skipping download.")
        return

    # If archives are present (e.g., from a previous partial download), try extracting them first
    if _any_archives_in_dir(source):
        print(f"Found archives for {dataset_name_for_print} in {source}; extracting...")
        extract_all_archives(source, dest)
        return
    else:
        print(f"No archives found for {dataset_name_for_print} in {source}; downloading...")
        if _any_images_in_dir(dest):
            print(f"{dataset_name_for_print} ready after extraction.")
            return
        if RUNNING_IN_COLAB:
            print("Running in Google Colab")
            # Check the environment variables exist
            kaggle_username = userdata.get('KAGGLE_USERNAME')
            kaggle_key = userdata.get('KAGGLE_KEY')
            if kaggle_username and kaggle_key:
                print("KAGGLE_USERNAME and KAGGLE_KEY environment variables are defined.")
            else:
                print("KAGGLE_USERNAME or KAGGLE_KEY environment variables are NOT defined.")
                if not kaggle_username:
                    print(" - KAGGLE_USERNAME is not defined.")
                if not kaggle_key:
                    print(" - KAGGLE_KEY is not defined.")
                return
        else:
            print("Not running in Google Colab")
            if not kaggle_creds.exists():
                print(f"Kaggle credentials missing; cannot download {dataset_name_for_print}.")
                return

        print(f"Downloading {dataset_name_for_print} via Kaggle: {dataset_slug}")
        cmd = f'kaggle datasets download -d {dataset_slug} -p "{dest}"'
        r = os.system(cmd)
        if r != 0:
            print(f"Kaggle download failed for {dataset_slug}.")
            return

        print(f"Extracting archive {source} to {dataset_name_for_print}")
        extract_all_archives(source, dest)

    if _any_images_in_dir(dest):
        print(f"Found existing {dataset_name_for_print} images in {dest}; skipping download.")
        return

```

```

        print(f"{{dataset_name_for_print}} ready.")
    else:
        print(f"No images detected for {{dataset_name_for_print}} after extraction. Check contents in {{dest}}.")

```

Other helper utils

```

def extract_archive(archive_path: Path, dest_dir: Path):
    dest_dir.mkdir(parents=True, exist_ok=True)
    ap = str(archive_path).lower()
    print(f"Extracting {{archive_path}} to {{dest_dir}}...")
    if ap.endswith(".zip"):
        with zipfile.ZipFile(archive_path, 'r') as zf: zf.extractall(dest_dir)
    elif ap.endswith(("tar", ".tar.gz", ".tgz", ".tar.bz2", ".tbz")):
        import tarfile
        with tarfile.open(archive_path, 'r:*') as tf: tf.extractall(dest_dir)
    else:
        raise ValueError(f"Unknown archive format: {{archive_path}}")
    print("Done extracting files.")

def find_images(root: Path, include_dirs=None):
    exts = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
    roots = [root]
    if include_dirs:
        roots = [root / d for d in include_dirs if (root / d).exists()] or [root]
    files = []
    for r in roots:
        for p in r.rglob("*"):
            if p.suffix.lower() in exts and p.is_file(): files.append(p)
    return files

def to_rgb(img: Image.Image) -> Image.Image:
    if img.mode in ("RGBA", "LA"):
        bg = Image.new("RGB", img.size, (255, 255, 255)); bg.paste(img, mask=img.split()[-1]); return bg
    return img.convert("RGB") if img.mode != "RGB" else img

def center_crop_resize(img: Image.Image, size=224) -> Image.Image:
    return ImageOps.fit(to_rgb(img), (size, size), method=Image.BILINEAR, bleed=0.0, centering=(0.5, 0.5))

def safe_save_jpeg(img: Image.Image, path: Path):
    path.parent.mkdir(parents=True, exist_ok=True)
    img.save(path, format="JPEG", quality=JPEG_QUALITY, optimize=True)

def phash_file(path: Path):
    try:
        with Image.open(path) as im: return imagehash.phash(to_rgb(im))
    except Exception: return None

def deduplicate_paths(paths, verbose_name="items"):
    seen, unique, dupes = {}, [], 0
    for p in tqdm(paths, desc=f"Dedup {{verbose_name}}", leave=False):
        h = phash_file(p)
        if h is None: continue
        if h in seen: dupes += 1; continue
        seen[h] = p; unique.append(p)
    if dupes: print(f"• Removed {{dupes}} duplicates (pHash) from {{verbose_name}}.")
    return unique

def check_image_rules(path: Path):
    """Return (ok: bool, reason: str). Applies only if APPLY_IMAGE_FILTERS=True."""
    try:
        with Image.open(path) as im:
            w, h = im.size
            short_side = min(w, h)
            ar = (w / h) if h else 0
    except Exception as e:
        return False, f"unreadable ({{e.__class__.__name__}})"

    if not APPLY_IMAGE_FILTERS:
        return True, ""

    if short_side < MIN_SHORT_SIDE_PX:
        return False, f"min-dim too small (min={{short_side}}px < {{MIN_SHORT_SIDE_PX}}px)"
    if ar < MIN_ASPECT_RATIO or ar > MAX_ASPECT_RATIO:
        return False, f"aspect ratio {{ar:.2f}} outside [{{MIN_ASPECT_RATIO:.2f}},{{MAX_ASPECT_RATIO:.2f}}]"
    return True, ""

```

```

def filter_paths_with_logging(paths, class_name: str):
    kept = []
    rejected_count = 0
    for p in tqdm(paths, desc=f"Filter images ({class_name})", leave=False):
        ok, reason = check_image_rules(p)
        if ok:
            kept.append(p)
        else:
            rejected_count += 1
            if PRINT_REJECTION_MESSAGES:
                print(f"Reject [{class_name}]: {p} → {reason}")
    if rejected_count > 0:
        print(f"• Rejected {rejected_count} images from {class_name} due to quality filters.")
    return kept

# ***** Talk about how this keeps the groups of humans balanced *****

def sample_n_stratified(df, group_cols, n_total, seed=SEED):
    groups = df.groupby(group_cols)
    combos = list(groups.groups.keys())
    k = len(combos)
    base_n = n_total // k
    remainder = n_total - base_n * k

    sampled = []
    capacities = []
    for key, g in groups:
        take = min(base_n, len(g))
        samp = g.sample(n=take, random_state=seed) if take > 0 else g.iloc[[]]
        sampled.append(samp)
        capacities.append((key, len(g) - take))
    result = pd.concat(sampled, ignore_index=True)

    if remainder > 0:
        capacities.sort(key=lambda x: x[1], reverse=True)
        i = 0
        while remainder > 0 and i < len(capacities):
            key, cap = capacities[i]
            if cap > 0:
                g = groups.get_group(key).drop(result.index, errors="ignore")
                take = min(1, len(g))
                if take > 0:
                    extra = g.sample(n=take, random_state=seed + remainder + i)
                    result = pd.concat([result, extra], ignore_index=True)
                    remainder -= take
            i += 1
        if remainder > 0:
            rest = df.drop(result.index, errors="ignore")
            extra = rest.sample(n=min(remainder, len(rest)), random_state=seed + 999)
            result = pd.concat([result, extra], ignore_index=True)

    return result.sample(frac=1.0, random_state=seed).reset_index(drop=True)

def split_dataframe_train_val_test(df: pd.DataFrame, split=SPLIT, seed=SEED):
    """Splits a DataFrame into train/val/test based on proportions and shuffles, preserving all columns."""
    n = len(df)
    n_train = int(n * split["train"])
    n_val = int(n * split["val"])

    # Shuffle the DataFrame and reset index
    shuffled_df = df.sample(frac=1.0, random_state=seed).reset_index(drop=True)

    # Get the DataFrame slices for each split
    df_train = shuffled_df[:n_train].copy()
    df_val = shuffled_df[n_train:n_train+n_val].copy()
    df_test = shuffled_df[n_train+n_val:].copy()

    return {"train": df_train,
            "val": df_val,
            "test": df_test}

def split_list_train_val_test(data: list, split=SPLIT, seed=SEED):
    """Splits a list into train/val/test based on proportions and shuffles."""
    n = len(data)

```

```

n_train = int(n * split["train"])
n_val = int(n * split["val"])

# Shuffle the list
shuffled_list = data[:] # Create a copy to avoid modifying the original list
random.Random(seed).shuffle(shuffled_list)

# Get the list slices for each split
list_train = shuffled_list[:n_train]
list_val = shuffled_list[n_train:n_train+n_val]
list_test = shuffled_list[n_train+n_val:]

return {"train": list_train,
        "val": list_val,
        "test": list_test}

def copy_and_process(df_by_split, class_name):
    processed_data = [] # List to store dictionaries of processed image info
    for split_name, df in df_by_split.items(): # Changed 'data' to 'df' here
        if isinstance(df, pd.DataFrame): # Changed 'data' to 'df' here
            # Process DataFrame (for human data)
            for index, row in tqdm(df.iterrows(), total=len(df), desc=f"Write {class_name}/{split_name}", leave=False):
                src = row["path"]
                new_filename = row["new_filename"] # Use the pre-calculated filename
                try:
                    with Image.open(src) as im:
                        # Safety: re-check rules in case something slipped through
                        ok, reason = check_image_rules(src)
                        if not ok:
                            if PRINT_REJECTION_MESSAGES:
                                print(f"Reject at write [{class_name}]: {src} → {reason}")
                                continue
                        out = center_crop_resize(im, IMG_SIZE)
                        out_path = OUT_DIR / split_name / class_name / new_filename
                        safe_save_jpeg(out, out_path)

                        # Append info to the list
                        processed_data.append({
                            "original_path": str(src),
                            "new_path": str(out_path),
                            "split": split_name,
                            "class": class_name,
                            "age": row.get("age"), # Use .get() in case age column doesn't exist for other classes
                            "gender": row.get("gender"),
                            "race": row.get("race"),
                        })

                except Exception as e:
                    if PRINT_REJECTION_MESSAGES:
                        print(f"Reject at write [{class_name}]: {src} → unreadable ({e.__class__.__name__})")
                        continue
            elif isinstance(df, list): # Changed 'data' to 'df' here
                # Process list of paths (for avatar and animal data)
                for src in tqdm(df, total=len(df), desc=f"Write {class_name}/{split_name}", leave=False):
                    try:
                        with Image.open(src) as im:
                            # Safety: re-check rules in case something slipped through
                            ok, reason = check_image_rules(src)
                            if not ok:
                                if PRINT_REJECTION_MESSAGES:
                                    print(f"Reject at write [{class_name}]: {src} → {reason}")
                                    continue
                                out = center_crop_resize(im, IMG_SIZE)
                            # Generate new filename for list-based processing
                            new_filename = hashlib.md5(str(src).encode()).hexdigest()[:16] + ".jpg"
                            out_path = OUT_DIR / split_name / class_name / new_filename
                            safe_save_jpeg(out, out_path)

                            # Append info to the list (without specific labels for non-human data)
                            processed_data.append({
                                "original_path": str(src),
                                "new_path": str(out_path),
                                "split": split_name,
                                "class": class_name,
                                "age": None,
                                "gender": None,
                            })

```



```

        "race": None,
    })

except Exception as e:
    if PRINT_REJECTION_MESSAGES:
        print(f"Reject at write [{class_name}]: {src} → unreadable ({e.__class__.__name__})")
    continue
return processed_data # Return the collected data

```

✓ 1) Download FairFace (padding=1.25) + labels

```

print("== Ensuring FairFace availability ==")
ensure_fairface_downloaded()

== Ensuring FairFace availability ==
FairFace not fully present; checking files...
Found FairFace image zip; skipping re-download.
Found FairFace label train CSV; skipping re-download.
Found FairFace label val CSV; skipping re-download.
Copying CSVs from /content/zip/fairface to /content/raw/fairface...
Copied fairface_label_train.csv
Copied fairface_label_val.csv
Extracting FairFace archives...
Extracting /content/zip/fairface/fairface-img-margin125-trainval.zip to /content/raw/fairface...
Done extracting files.
FairFace ready.

```

✓ 2) Parse labels, 'working age' filter, build strata

Here we choose the human images that will be included in the dataset. The key constraint, working age, is used to exclude images where the human is not likely to be in the work force. This explicitly includes individuals in the age groups **'20-29', '30-39', '40-49', '50-59', and '60-69'** and excludes individuals in the **'0-2', '3-9', '10-19', and 'more than 70'** age groups, ensuring that no one under 20 or over 69 is included in the filtered FairFace data used for the dataset.

```

import pandas as pd
from IPython.display import HTML, display

def load_fairface_df(ff_dir: Path):
    df_train = pd.read_csv(next(ff_dir.glob("*label_train*.csv")))
    df_val = pd.read_csv(next(ff_dir.glob("*label_val*.csv")))

    # Add a 'split' column to indicate the original source
    df_train['original_split'] = 'train'
    df_val['original_split'] = 'val'

    df = pd.concat([df_train, df_val], ignore_index=True)

    cols = {c.lower().strip(): c for c in df.columns}
    def map_col(*cands):
        for c in cands:
            lc = c.lower().strip()
            if lc in cols: return cols[lc]
        for k in cols:
            if any(cc in k for cc in cands): return cols[k]
        return None
    file_col = map_col("file", "image", "path")
    age_col = map_col("age")
    gender_col = map_col("gender", "sex")
    race_col = map_col("race", "ethnicity", "race_7", "race_8", "race_4")
    split_col = map_col("original_split")

    if any(x is None for x in [file_col, age_col, gender_col, race_col, split_col]):
        missing = [name for name, col in [("file", file_col), ("age", age_col), ("gender", gender_col), ("race", race_col), ("ori
        raise ValueError(f"Could not map needed columns. Missing: {missing}. Found: {list(df.columns)}")

```

```

df = df[[file_col, age_col, gender_col, race_col, split_col]].rename(
    columns={file_col:"file", age_col:"age", gender_col:"gender", race_col:"race", split_col:"original_split"}
)

# Resolve image paths inside extracted pad=1.25 pack (usually under train/ and val/)
img_root = ff_dir
def resolve_path(rel):
    rel = str(rel).lstrip("./\\")
    cand = img_root / rel
    if cand.exists(): return cand
    for sub in ["train", "val", "fairface-img-margin125-trainval/train", "fairface-img-margin125-trainval/val"]:
        c = img_root / sub / rel
        if c.exists(): return c
    hits = list(img_root.rglob(Path(rel).name))
    return hits[0] if hits else None

df["path"] = df["file"].apply(resolve_path)
df = df[df["path"].notna()].reset_index(drop=True)

# Add a column for the new filename (based on hash)
df["new_filename"] = df["path"].apply(lambda p: hashlib.md5(str(p).encode()).hexdigest()[:16] + ".jpg")

# Get age distribution before adult filtering
age_counts_before = df['age'].value_counts().sort_index() # Sort by age group index

# Print unique age groups before adult filter
# print("\nUnique age groups before adult filter:")
# display(age_counts_before) # Removed individual display
# print("\nData type of age column before adult filter:", df['age'].dtype)
# print("\nUnique age values and their types before adult filter:")
# for val in df['age'].unique():
#     print(f"Value: '{val}', Type: {type(val)}")

# Adult-only: explicitly include known adult age bins
adult_age_groups = {'20-29', '30-39', '40-49', '50-59', '60-69'} # Excluded 'more than 70'

for col in ["age", "gender", "race"]:
    df[col] = df[col].astype(str).str.strip()

# Filter to keep only rows where 'age' is in the adult_age_groups set
df = df[df["age"].isin(adult_age_groups)].reset_index(drop=True)

# Get age distribution after adult filtering
age_counts_after = df['age'].value_counts()

# Print age distribution after adult filtering
# print("\nAge distribution after adult filter:")
# display(age_counts_after) # Removed individual display

# Display tables side by side
html_output = f"""
<div style="display: flex;">
    <div style="margin-right: 20px;">
        <h3>Age Distribution Before Filter</h3>
        {age_counts_before.to_frame().to_html()}
    </div>
    <div>
        <h3>Age Distribution After Filter</h3>
        {age_counts_after.to_frame().to_html()}
    </div>
</div>
"""
display(HTML(html_output))

# (race × gender × adult-age-bin) strata to preserve FairFace balance in adult subset
df["strata"] = df["race"] + " | " + df["gender"] + " | " + df["age"]
return df

fairface_df = load_fairface_df(FAIRFACE_RAW_DIR)
print("\nFairFace (adult) candidates (pre-filter):", len(fairface_df), " strata:", len(fairface_df["strata"].unique()))

# Apply image-quality filters to FairFace before stratified sampling (maintains balance within the *filtered* adult pool)

```

```

ff_keep_mask = []
for row in tqdm(fairface_df.itertuples(index=False), total=len(fairface_df), desc="Filter images (human/FairFace)", leave=False):
    ok, reason = check_image_rules(row.path)
    ff_keep_mask.append(ok)
    if not ok and PRINT_REJECTION_MESSAGES:
        print(f"Reject [human]: {row.path} → {reason}")
fairface_df = fairface_df[np.array(ff_keep_mask, dtype=bool)].reset_index(drop=True)
print("FairFace (adult) candidates (post-filter):", len(fairface_df))

```

Age Distribution Before Filter Age Distribution After Filter

	count		count
age		age	
0-2	1991	20-29	28898
10-19	10284	30-39	21580
20-29	28898	40-49	12097
3-9	11764	50-59	7024
30-39	21580	60-69	3100
40-49	12097		
50-59	7024		
60-69	3100		
more than 70	960		

FairFace (adult) candidates (pre-filter): 72699 strata: 70
FairFace (adult) candidates (post-filter): 72699

✓ 3) Stratified sample N=TARGET_PER_CLASS humans

- Select a subset of human images.
- The deduplicate_paths function processes a list of image file paths to find and remove duplicates. It does this by calculating a "perceptual hash" for each image, which represents its visual characteristics. If two images have the same hash, they are considered duplicates, and only the first one encountered is kept. The function returns a new list containing only the unique image paths.
- Aim for a total of TARGET_PER_CLASS images.
- Use a stratified sampling method to ensure the sample is representative of the different demographic groups (race, gender, age) present in the filtered FairFace data.

```

if len(fairface_df) < MIN_REQUIRED_PER_CLASS:
    raise RuntimeError(f"Not enough adult FairFace images ({len(fairface_df)}) to sample {TARGET_PER_CLASS}.")

# Deduplicate fairface_df BEFORE sampling
print("Deduplicating fairface_df before sampling...")
original_count = len(fairface_df)
unique_paths = deduplicate_paths(fairface_df["path"].tolist(), "fairface (pre-sample)")
unique_paths_str = set(str(p) for p in unique_paths)
fairface_df_deduped = fairface_df[fairface_df["path"].apply(lambda x: str(x) in unique_paths_str)].reset_index(drop=True)
deduplicated_count = len(fairface_df_deduped)
print(f"Removed {original_count - deduplicated_count} duplicate images from fairface_df.")
print("FairFace (adult, deduped) candidates:", len(fairface_df_deduped))

# Now sample from the deduplicated DataFrame
if len(fairface_df_deduped) < MIN_REQUIRED_PER_CLASS:
    raise RuntimeError(f"Not enough adult FairFace images ({len(fairface_df_deduped)}) after deduplication to sample {TARGET_PER_CLASS}")

human_sample = sample_n_stratified(fairface_df_deduped, ["strata"], TARGET_PER_CLASS, seed=SEED)

# Re-add the new_filename column to human_sample after sampling if it was lost (sample_n_stratified should preserve it, but as a s
if "new_filename" not in human_sample.columns:
    human_sample["new_filename"] = human_sample["path"].apply(lambda p: hashlib.md5(str(p).encode()).hexdigest()[:16] + ".jpg")

# Implement top-up logic if the initial sample is less than TARGET_PER_CLASS
if len(human_sample) < TARGET_PER_CLASS:

```

```

print(f"Initial stratified sample size ({len(human_sample)}) is less than target ({TARGET_PER_CLASS}). Topping up...")
# Identify images already in human_sample
chosen_paths = set(human_sample["path"].apply(str))
# Get remaining deduplicated images not in human_sample
remaining_df = fairface_df_deduped[~fairface_df_deduped["path"].apply(str).isin(chosen_paths)].copy()

need = TARGET_PER_CLASS - len(human_sample)
if len(remaining_df) >= need:
    print(f"Sampling {need} additional images from remaining pool.")
    topup_sample = remaining_df.sample(n=need, random_state=SEED + 1000)
    human_sample = pd.concat([human_sample, topup_sample], ignore_index=True)
    # Ensure new_filename is present in topup_sample if needed
    if "new_filename" not in topup_sample.columns:
        topup_sample["new_filename"] = topup_sample["path"].apply(lambda p: hashlib.md5(str(p).encode()).hexdigest()[:16] + '
        # Need to update human_sample if topup_sample got a new column
        human_sample = pd.concat([human_sample, topup_sample], ignore_index=True) # Re-concat with new column
        human_sample = human_sample.loc[~human_sample.index.duplicated(keep='last')] # Remove duplicates introduced by re-cor
    human_sample = pd.concat([human_sample, topup_sample], ignore_index=True)
    human_sample = human_sample.drop_duplicates(subset=["path"]).reset_index(drop=True) # Deduplicate based on paths after cor

else:
    print(f"Warning: Not enough remaining images ({len(remaining_df)}) to reach target ({TARGET_PER_CLASS}). Sample size will
    # Take all remaining images if not enough to meet the need
    human_sample = pd.concat([human_sample, remaining_df], ignore_index=True)
    # Ensure new_filename is present in remaining_df if needed
    if "new_filename" not in remaining_df.columns:
        remaining_df["new_filename"] = remaining_df["path"].apply(lambda p: hashlib.md5(str(p).encode()).hexdigest()[:16] + '
        # Need to update human_sample if remaining_df got a new column
        human_sample = pd.concat([human_sample, remaining_df], ignore_index=True) # Re-concat with new column
        human_sample = human_sample.loc[~human_sample.index.duplicated(keep='last')] # Remove duplicates introduced by re-cor

    human_sample = pd.concat([human_sample, remaining_df], ignore_index=True)
    human_sample = human_sample.drop_duplicates(subset=["path"]).reset_index(drop=True) # Deduplicate based on paths after cor

# After potential top-up, ensure the new_filename column is present and unique
if "new_filename" not in human_sample.columns:
    human_sample["new_filename"] = human_sample["path"].apply(lambda p: hashlib.md5(str(p).encode()).hexdigest()[:16] + ".jpg")

# Remove duplicates based on new_filename after any concatenation
human_sample = human_sample.drop_duplicates(subset=["new_filename"]).reset_index(drop=True)

print("Human sample size after potential top-up:", len(human_sample))

# Note: The final number of images written to the ImageFolder might still be less than TARGET_PER_CLASS
# due to the image quality filters applied during the copy_and_process step.
# The user's request to have the *final* counts equal requires a different approach (oversampling + undersampling after filtering)
# For now, we are ensuring the sample *entering* the processing stage is TARGET_PER_CLASS if possible.

```

Deduplicating fairface_df before sampling...

Dedup fairface (pre-sample): 38%

27967/72699 [01:25<02:26, 305.39it/s]

4) Kaggle downloads: Avatars + Animal faces

```

AVATAR_DIR = RAW_DIR / "avatars"
ANIMAL_DIR = RAW_DIR / "animal_faces"
print("\n== Ensuring Kaggle datasets availability ==")
ensure_kaggle_downloaded(
    KAGGLE_AVATARS_DATASET,
    ZIP_DIR / "avatars",
    RAW_DIR / "avatars",
    "Avatars (Google Cartoon Set)")
ensure_kaggle_downloaded(
    KAGGLE_ANIMALFACES_DATASET,
    ZIP_DIR / "animal_faces",
    RAW_DIR / "animal_faces",
    "Animal Faces (Dogs vs Cats)")

```

✓ 5) Collect & sample Avatars (with filters + dedup)

```
avatar_all = find_images(AVATAR_DIR, include_dirs=AVATAR_INCLUDE_DIRS)
avatar_all = filter_paths_with_logging(avatar_all, "avatar")
avatar_all = deduplicate_paths(avatar_all, "avatar")
if len(avatar_all) < MIN_REQUIRED_PER_CLASS:
    raise RuntimeError(f"Not enough avatar images ({len(avatar_all)}) for {TARGET_PER_CLASS}.")
avatar_paths = random.sample(avatar_all, TARGET_PER_CLASS)
```

✓ 6) Collect & sample Animal faces (with filters + dedup)

```
animal_all = find_images(ANIMAL_DIR, include_dirs=["test", "train"])
print(f"ANIMAL_DIR {ANIMAL_DIR}")
print(f"Animal Images found: {len(animal_all)}")
animal_all = filter_paths_with_logging(animal_all, "animal")
animal_all = deduplicate_paths(animal_all, "animal")
if len(animal_all) < MIN_REQUIRED_PER_CLASS:
    raise RuntimeError(f"Not enough animal images ({len(animal_all)}) for {TARGET_PER_CLASS}.")
animal_paths = random.sample(animal_all, TARGET_PER_CLASS)
print(len(animal_paths))
```

✓ 7) Split & write ImageFolder with resize/crop

```
# if OUT_DIR.exists():
#     print("Clearing old output:", OUT_DIR); shutil.rmtree(OUT_DIR)
OUT_DIR.mkdir(parents=True, exist_ok=True)

# Clear only the image subdirectories within OUT_DIR to preserve CSVs and manifest
print("Clearing old image subdirectories in:", OUT_DIR)
for split_name in ["train", "val", "test"]:
    split_dir = OUT_DIR / split_name
    for cls in ["human", "avatar", "animal"]:
        class_dir = split_dir / cls
        if class_dir.exists():
            print(f" Removing: {class_dir}")
            shutil.rmtree(class_dir)

# Use the DataFrame for human data to preserve labels
spl_human = split_dataframe_train_val_test(human_sample, SPLIT, SEED)
# Use lists of paths for avatar and animal as we are not preserving specific labels for them
spl_avatar = split_list_train_val_test/avatar_paths, SPLIT, SEED)
spl_animal = split_list_train_val_test(animal_paths, SPLIT, SEED)

processed_human_data = copy_and_process(spl_human, "human")
copy_and_process(spl_avatar, "avatar")
copy_and_process(spl_animal, "animal")

def count_images(root): return sum(1 for _ in root.rglob("*.jpg"))
for split_name in ["train", "val", "test"]:
    for cls in ["human", "avatar", "animal"]:
        print(f"{split_name:5s}/{cls:7s}: {count_images(OUT_DIR / split_name / cls)}")
```

Human Metadata

```
import pandas as pd

# Create a DataFrame from the processed human data
human_labels_df = pd.DataFrame(processed_human_data)

# Define the path for the new CSV file
human_labels_csv_path = OUT_DIR / "human_labels.csv"

# Save the DataFrame to a CSV file
human_labels_df.to_csv(human_labels_csv_path, index=False)

print(f"CSV with human labels and new paths saved to: {human_labels_csv_path}")

# Display the first few rows of the new CSV
display(human_labels_df.head())
```

Create CSV with Dataset Information

```
# fairface_csvs = [FAIRFACE_RAW_DIR / "fairface_label_train.csv", FAIRFACE_RAW_DIR / "fairface_label_val.csv"]
```

Is this the one we use?

```
# import pandas as pd

# # Load the train and validation FairFace CSVs
# df_train = pd.read_csv(FAIRFACE_RAW_DIR / "fairface_label_train.csv")
# df_val = pd.read_csv(FAIRFACE_RAW_DIR / "fairface_label_val.csv")

# # Concatenate the two DataFrames
# fairface_combined_df = pd.concat([df_train, df_val], ignore_index=True)

# # Resolve image paths inside extracted pad=1.25 pack (usually under train/ and val/)
# img_root = FAIRFACE_RAW_DIR
# def resolve_path(rel):
#     rel = str(rel).rstrip("./\\")
#     cand = img_root / rel
#     if cand.exists(): return cand
#     for sub in ["train", "val", "fairface-img-margin125-trainval/train", "fairface-img-margin125-trainval/val"]:
#         c = img_root / sub / rel
#         if c.exists(): return c
#     hits = list(img_root.rglob(Path(rel).name))
#     return hits[0] if hits else None

# fairface_combined_df["path"] = fairface_combined_df["file"].apply(resolve_path)
# fairface_combined_df = fairface_combined_df[fairface_combined_df["path"].notna()].reset_index(drop=True)

# # ***** Why are we deduping here? *****

# # Deduplicate based on image content using phash
# # print("Deduplicating combined FairFace DataFrame...")
# # original_count = len(fairface_combined_df)
# # unique_paths = deduplicate_paths(fairface_combined_df["path"].tolist(), "combined FairFace")
# # unique_paths_str = set(str(p) for p in unique_paths)
# # fairface_combined_df = fairface_combined_df[fairface_combined_df["path"].apply(lambda x: str(x) in unique_paths_str)].reset_index(drop=True)
# # deduplicated_count = len(fairface_combined_df)
```

```
# # print(f"Removed {original_count - deduplicated_count} duplicate images.")

# # Define the path for the new combined CSV file
# combined_csv_path = OUT_DIR / "fairface_combined_labels.csv"

# # Save the combined DataFrame to a new CSV file in the OUT_DIR
# fairface_combined_df.to_csv(combined_csv_path, index=False)

# print(f"Combined FairFace CSV saved to: {combined_csv_path}")
# display(fairface_combined_df.head())
```

✓ Is this a dup

```
# # Copy FairFace CSVs to OUT_DIR for completeness
# print("\nCopying FairFace CSVs to OUT_DIR...")
# fairface_csvs = [FAIRFACE_RAW_DIR / "fairface_label_train.csv", FAIRFACE_RAW_DIR / "fairface_label_val.csv"]
# for csv_file in fairface_csvs:
#     if csv_file.exists():
#         shutil.copy(csv_file, OUT_DIR)
#         print(f"Copied {csv_file.name} to {OUT_DIR}")
#     else:
#         print(f"Warning: {csv_file.name} not found in {FAIRFACE_RAW_DIR}. Skipping copy.")
```

✓ 8) Manifest for traceability

```
import json

# Function to count images in a class directory within a split
def count_images_in_split_class(split_dir: Path, class_name: str):
    class_path = split_dir / class_name
    if class_path.exists():
        return sum(1 for _ in class_path.rglob("*.jpg"))
    return 0

# Get final counts from the output directory
final_counts = {}
for split_name in ["train", "val", "test"]:
    final_counts[split_name] = {}
    split_dir = OUT_DIR / split_name
    for cls in ["human", "avatar", "animal"]:
        final_counts[split_name][cls] = count_images_in_split_class(split_dir, cls)

# Calculate total counts per class across all splits
total_counts_per_class = {
    cls: sum(final_counts[split_name][cls] for split_name in ["train", "val", "test"])
    for cls in ["human", "avatar", "animal"]
}

manifest = {
    "seed": SEED,
    "img_size": IMG_SIZE,
    "final_image_counts_per_split_and_class": final_counts,
    "total_image_counts_per_class": total_counts_per_class,
    "split_proportions": SPLIT,
    "sources": {
        "fairface_padding": "1.25",
        "fairface_readme": "https://github.com/joojs/fairface",
        "fairface_img_id": FAIRFACE_IMG_PAD125_ID,
        "fairface_label_train_id": FAIRFACE_LABEL_TRAIN_ID,
        "fairface_label_val_id": FAIRFACE_LABEL_VAL_ID,
        "kaggle_avatars": KAGGLE_AVATARS_DATASET,
        "kaggle_animals": KAGGLE_ANIMALFACES_DATASET,
        "avatar_include_dirs": AVATAR_INCLUDE_DIRS,
        "animal_include_dirs": ANIMAL_INCLUDE_DIRS,
    },
    "target_per_class_pre_filter": TARGET_PER_CLASS, # Note: This is the target before final filtering
    "filters": {
        "apply": APPLY_IMAGE_FILTERS,
        "min_short_side_px": MIN_SHORT_SIDE_PX,
        "min_ar": MIN_ASPECT_RATIO,
```



```

        "max_ar": MAX_ASPECT_RATIO,
        "print_rejections": PRINT_REJECTION_MESSAGES,
    },
}
(OUT_DIR / "MANIFEST.json").write_text(json.dumps(manifest, indent=2))
print("\nDataset build complete.")
print("Output (ImageFolder):", OUT_DIR.resolve())

```

```

end_time = time.time()
print(f"Run started at: {time.ctime(start_time)}")
print(f"Run ended at: {time.ctime(end_time)}")
elapsed_time_seconds = end_time - start_time
elapsed_time_minutes = elapsed_time_seconds / 60
print(f"Total elapsed time: {elapsed_time_minutes:.2f} minutes")

```

✓ Test code - remove

```

# # Filter the DataFrame for the '20-29' age group
# age_20_29_df = human_labels_df[human_labels_df['age'] == '20-29']

# # Get the count of individuals in this age group
# count_20_29 = len(age_20_29_df)

# print(f"Number of individuals in the '20-29' age group: {count_20_29}")

```

✓ Test code - remove

```

# # Show breakdown of age distribution per split for human data
# if 'human_labels_df' in locals() and not human_labels_df.empty:
#     print("\nAge distribution breakdown for human dataset by split:")
#     age_split_breakdown = human_labels_df.groupby(['split', 'age']).size().unstack(fill_value=0)
#     display(age_split_breakdown)
# else:
#     print("\nhuman_labels_df not found or is empty. Please run preceding cells to generate the human dataset and labels CSV.")

```

✓ Test code - remove

```

# # Check if '20-29' age group exists in fairface_df and count
# if 'age' in fairface_df.columns:
#     count_20_29_fairface_df = fairface_df[faiface_df['age'] == '20-29'].shape[0]
#     print(f"Number of individuals in '20-29' age group in fairface_df (after initial adult filter): {count_20_29_fairface_df}")
# else:
#     print("'age' column not found in fairface_df.")

# # Also check in human_sample (after sampling, before splitting)
# if 'age' in human_sample.columns:
#     count_20_29_human_sample = human_sample[human_sample['age'] == '20-29'].shape[0]
#     print(f"Number of individuals in '20-29' age group in human_sample (after sampling): {count_20_29_human_sample}")
# else:
#     print("'age' column not found in human_sample.")

```

✓ Tools

```

def gcs_auth():
    import json
    from google.colab import userdata

    # Get the service account key from Colab Secrets

```

```

service_account_info = json.loads(userdata.get('GCP_SERVICE_ACCOUNT_KEY'))

# Define the path to save the service account key file
key_file_path = 'service_account_key.json'

# Save the service account key to a file
with open(key_file_path, 'w') as f:
    json.dump(service_account_info, f)

# Authenticate gcloud and gsutil using the service account key file
!gcloud auth activate-service-account --key-file {key_file_path}

```

```

def load_zip_data_from_gcs():
    import time

    start_time = time.time()

    bucket_name = 'gs://along-capstone-data'
    source_directory = 'zip' # This is the directory in the bucket
    destination_directory = '.' # This is the local destination directory

    gcs_auth()

    # Copy the directory containing the zip file
    !gsutil -m -q cp -r {bucket_name}/{source_directory} {destination_directory}

    # # Unzip the data directory.
    # print("Unzipping data...")
    # !unzip -o -q {local_zip_file_path} -d {destination_directory}

    end_time = time.time()
    elapsed_time_seconds = end_time - start_time
    elapsed_time_minutes = int(elapsed_time_seconds // 60)
    elapsed_time_remaining_seconds = int(elapsed_time_seconds % 60)

    print(f"Zip download and unzip elapsed time: {elapsed_time_minutes} minutes and {elapsed_time_remaining_seconds} seconds")

# load_zip_data_from_gcs()

```

```

def save_data_to_gcs():
    import time

    start_time = time.time()

    bucket_name = 'gs://along-capstone-data'
    destination_directory = 'final'
    source_directory = 'final'

    gcs_auth()

    !gsutil -m -q cp -r {source_directory} {bucket_name}/{destination_directory}

    end_time = time.time()
    elapsed_time_seconds = end_time - start_time
    elapsed_time_minutes = int(elapsed_time_seconds // 60)
    elapsed_time_remaining_seconds = int(elapsed_time_seconds % 60)

    print(f"Upload elapsed time: {elapsed_time_minutes} minutes and {elapsed_time_remaining_seconds} seconds")

# save_data_to_gcs()

```

```

def save_final_zip_data_to_gcs():
    import time

    start_time = time.time()

    bucket_name = 'gs://along-capstone-data'
    destination_directory = 'final_zip'
    source_file = 'final.zip' # Name of the zip file to create

    gcs_auth()

    # Zip up the data directory.
    print("Zipping data...")

```

```

!zip -r -q {source_file} final

!gsutil -m -q cp -r {source_file} {bucket_name}/{destination_directory}

end_time = time.time()
elapsed_time_seconds = end_time - start_time
elapsed_time_minutes = int(elapsed_time_seconds // 60)
elapsed_time_remaining_seconds = int(elapsed_time_seconds % 60)

print("Upload complete.")
print(f"Upload elapsed time: {elapsed_time_minutes} minutes and {elapsed_time_remaining_seconds} seconds")

# save_zip_data_to_gcs()

```

✓ Probably not needed in this notebook

✓ Cleanup Utilities - Probably Delete

```

# def _is_subpath(child: Path, parent: Path) -> bool:
#     try:
#         child.resolve().relative_to(parent.resolve())
#         return True
#     except Exception:
#         return False

# def _safe_rmtree(p: Path):
#     if p.exists():
#         if _is_subpath(p, WORK_DIR):
#             shutil.rmtree(p)
#             print(f"Removed: {p}")
#         else:
#             print(f"Skip delete (outside WORK_DIR): {p}")

# def cleanup_generated_dataset():
#     """Remove only the generated ImageFolder dataset (data/final)."""
#     _safe_rmtree(OUT_DIR)

# def cleanup_all_datasets():
#     """Remove initial raw downloads (data/raw) and the generated dataset (data/final)."""
#     _safe_rmtree(OUT_DIR)
#     _safe_rmtree(RAW_DIR)
#     _safe_rmtree(TMP_DIR)

# def cleanup_datasets(mode: str = "generated"):
#     """
#     mode ∈ {"generated", "all"}
#     - "generated": remove only data/final
#     - "all":      remove data/raw, data/final, data/tmp
#     """
#     mode = mode.lower().strip()
#     if mode == "generated":
#         cleanup_generated_dataset()
#     elif mode == "all":
#         cleanup_all_datasets()
#     else:
#         raise ValueError("mode must be 'generated' or 'all'")

```

```

# try:
#     drive.mount('/content/drive')
# except Exception as e:
#     print(f"Error mounting Google Drive: {e}")

```

```

# def copy_data_to_drive():
#     """Copies the data directory from Colab to Google Drive."""
#     if not COLAB_DATA_DIR.exists():
#         print(f"Source directory not found: {COLAB_DATA_DIR}")
#         return

#     DRIVE_DATA_PARENT_DIR.mkdir(parents=True, exist_ok=True)

#     # Remove existing directory in Drive to avoid errors during copy

```

```

# if DRIVE_DATA_DIR.exists():
#     print(f"Removing existing data directory in Drive: {DRIVE_DATA_DIR}")
#     shutil.rmtree(DRIVE_DATA_DIR)

# print(f"Copying data from {COLAB_DATA_DIR} to {DRIVE_DATA_DIR}...")
# try:
#     shutil.copytree(COLAB_DATA_DIR, DRIVE_DATA_DIR)
#     print("Data successfully copied to Google Drive.")
# except Exception as e:
#     print(f"Error copying data to Drive: {e}")

# def copy_data_from_drive():
#     """Copies the data directory from Google Drive to Colab."""
#     if not DRIVE_DATA_DIR.exists():
#         print(f"Source directory not found in Drive: {DRIVE_DATA_DIR}")
#         print("Please ensure Google Drive is mounted and the data exists at the specified path.")
#         return

#     # Remove existing directory in Colab to avoid errors during copy
#     if COLAB_DATA_DIR.exists():
#         print(f"Removing existing data directory in Colab: {COLAB_DATA_DIR}")
#         shutil.rmtree(COLAB_DATA_DIR)

#     print(f"Copying data from {DRIVE_DATA_DIR} to {COLAB_DATA_DIR}...")
#     try:
#         shutil.copytree(DRIVE_DATA_DIR, COLAB_DATA_DIR)
#         print("Data successfully copied to Colab environment.")
#     except Exception as e:
#         print(f"Error copying data from Drive: {e}")

```

```

# copy_data_to_drive()
# copy_data_from_drive()

```

```

# import shutil
# import os

# def zip_data_directory():
#     """Zips the 'data' directory."""
#     data_dir = Path("data")
#     if not data_dir.exists():
#         print(f"Error: Directory '{data_dir}' not found.")
#         return
#     zip_filename = data_dir.with_suffix(".zip")
#     print(f"Creating zip archive: {zip_filename} from {data_dir}")
#     try:
#         shutil.make_archive(str(data_dir), 'zip', data_dir)
#         print("Zip archive created successfully.")
#     except Exception as e:
#         print(f"Error zipping directory: {e}")

# def unzip_data_directory():
#     """Unzips the 'data.zip' archive."""
#     zip_filename = Path("data.zip")
#     if not zip_filename.exists():
#         print(f"Error: Zip file '{zip_filename}' not found.")
#         return
#     extract_dir = Path("data")
#     if extract_dir.exists():
#         print(f"Warning: Directory '{extract_dir}' already exists. Contents may be overwritten.")
#     print(f"Extracting zip archive: {zip_filename} to {extract_dir}")
#     try:
#         shutil.unpack_archive(str(zip_filename), str(extract_dir), 'zip')
#         print("Zip archive extracted successfully.")
#     except Exception as e:
#         print(f"Error unzipping archive: {e}")

```

```

# import shutil
# from pathlib import Path

# ZIP_NAME = "data.zip"

# # Define the source and destination paths
# source_zip = DRIVE_DATA_DIR / ZIP_NAME
# # Ensure the parent directory in Drive exists
# DRIVE_DATA_PARENT_DIR.mkdir(parents=True, exist_ok=True)

```

```

# # Define the destination path in Drive for the zip file
# drive_zip_destination = Path(ZIP_NAME)

# if not source_zip.exists():
#     print(f"Error: Source zip file '{source_zip}' not found.")
# else:
#     print(f"Copying '{source_zip}' to '{drive_zip_destination}'...")
#     try:
#         # Use shutil.copy2 to preserve metadata (optional, but good practice)
#         shutil.copy2(source_zip, drive_zip_destination)
#         unzip_data_directory()
#         print("Zip file successfully copied from Google Drive and unzipped.")
#     except Exception as e:
#         print(f"Error copying zip file to Drive: {e}")

```

```

# import shutil
# from pathlib import Path

# # zip_data_directory()

# # Define the source and destination paths
# source_zip = Path("data.zip")
# # Ensure the parent directory in Drive exists
# DRIVE_DATA_PARENT_DIR.mkdir(parents=True, exist_ok=True)
# # Define the destination path in Drive for the zip file
# drive_zip_destination = DRIVE_DATA_DIR / source_zip.name

# # Ensure the destination directory in Drive exists
# drive_zip_destination.parent.mkdir(parents=True, exist_ok=True)

# if not source_zip.exists():
#     print(f"Error: Source zip file '{source_zip}' not found.")
# else:
#     print(f"Copying '{source_zip}' to '{drive_zip_destination}'...")
#     try:
#         # Use shutil.copy2 to preserve metadata (optional, but good practice)
#         shutil.copy2(source_zip, drive_zip_destination)
#         print("Zip file successfully copied to Google Drive.")
#     except Exception as e:
#         print(f"Error copying zip file to Drive: {e}")

```

```

# cleanup_all_datasets()

```

✓ Task

Update the notebook to create a CSV file in the `OUT_DIR` that contains the new image paths generated by the `copy_and_process` function and the corresponding FairFace labels (age, gender, race) for those images.

✓ Modify `copy_and_process`

Subtask:

Update the `copy_and_process` function to return a list of dictionaries containing the new output path and the original row data for each successfully processed human image.

Reasoning: Modify the `copy_and_process` function to accept `return_labels` and the original DataFrame, and collect the processed image data with labels.

✓ Task

Modify the notebook to create a single CSV file named "dataset_manifest.csv" in the `OUT_DIR` that contains the columns 'split', 'label', and 'path' from the processed data of all classes (human, avatar, and animal).

Collect processed data for all classes

Subtask:

Modify cell `13e6dbd4-129e-4367-9e19-ceb6daf70d78` to store the returned list of dictionaries from `copy_and_process` for all three classes (human, avatar, animal).

Reasoning: The subtask is to modify the existing code cell to capture the output of `copy_and_process` for all classes. The current code already captures the human data. I will modify the calls for avatar and animal data to also capture their returned lists.

```
# if OUT_DIR.exists():
#     print("Clearing old output:", OUT_DIR); shutil.rmtree(OUT_DIR)
OUT_DIR.mkdir(parents=True, exist_ok=True)

# Clear only the image subdirectories within OUT_DIR to preserve CSVs and manifest
print("Clearing old image subdirectories in:", OUT_DIR)
for split_name in ["train", "val", "test"]:
    split_dir = OUT_DIR / split_name
    for cls in ["human", "avatar", "animal"]:
        class_dir = split_dir / cls
        if class_dir.exists():
            print(f" Removing: {class_dir}")
            shutil.rmtree(class_dir)

# Use the DataFrame for human data to preserve labels
spl_human = split_dataframe_train_val_test(human_sample, SPLIT, SEED)
# Use lists of paths for avatar and animal as we are not preserving specific labels for them
spl_avatar = split_list_train_val_test(avatar_paths, SPLIT, SEED)
spl_animal = split_list_train_val_test(animal_paths, SPLIT, SEED)

processed_human_data = copy_and_process(spl_human, "human")
processed_avatar_data = copy_and_process(spl_avatar, "avatar")
processed_animal_data = copy_and_process(spl_animal, "animal")

def count_images(root): return sum(1 for _ in root.rglob("*.jpg"))
for split_name in ["train", "val", "test"]:
    for cls in ["human", "avatar", "animal"]:
        print(f"{split_name:5s}/{cls:7s}: {count_images(OUT_DIR / split_name / cls)}")
```

Combine processed data

Subtask:

Combine the lists of dictionaries from all classes into a single list.

Reasoning: Combine the lists of dictionaries from all classes into a single list to prepare for creating a single manifest DataFrame.

```
all_processed_data = []
all_processed_data.extend(processed_human_data)
all_processed_data.extend(processed_avatar_data)
all_processed_data.extend(processed_animal_data)
```

✓ Create combined dataframe

Subtask:

Convert the combined list of dictionaries into a pandas DataFrame.

Reasoning: Create a pandas DataFrame from the combined list of processed data.

```
dataset_manifest_df = pd.DataFrame(all_processed_data)
```

✓ Rename and select columns

Subtask:

Rename the columns in the combined DataFrame to match the required names ('new_path' to 'path', 'class' to 'label') and select only the 'split', 'label', and 'path' columns.

Reasoning: Rename the columns and select the required columns to create the final dataset manifest DataFrame.

```
dataset_manifest_df = dataset_manifest_df.rename(columns={'new_path': 'path', 'class': 'label'})
dataset_manifest_df = dataset_manifest_df[['split', 'label', 'path']]
```

✓ Save combined manifest csv

Subtask:

Save the resulting DataFrame to a new CSV file (e.g., dataset_manifest.csv) in the OUT_DIR.

Reasoning: Save the combined and filtered DataFrame to a CSV file in the output directory.

```
dataset_manifest_csv_path = OUT_DIR / "dataset_manifest.csv"
dataset_manifest_df.to_csv(dataset_manifest_csv_path, index=False)
print(f"Dataset manifest CSV saved to: {dataset_manifest_csv_path}")
```

✓ Review existing csvs

Subtask:

Decide whether the previously generated CSVs (human_labels.csv and fairface_combined_labels.csv) are still needed or can be removed.

Reasoning: Examine the contents of human_labels_df and fairface_combined_df and compare them with dataset_manifest_df to determine if they are redundant or provide additional value. Then, add comments to the notebook indicating whether these files should be kept or can be safely removed.

```
print("\nComparing CSV contents:")

print("\nContents of human_labels_df:")
display(human_labels_df.head())
print("\nColumns in human_labels_df:", human_labels_df.columns.tolist())

# Check if fairface_combined_df exists before displaying
if 'fairface_combined_df' in locals():
    print("\nContents of fairface_combined_df:")
    display(fairface_combined_df.head())
    print("\nColumns in fairface_combined_df:", fairface_combined_df.columns.tolist())
else:
    print("\nfairface_combined_df was not created in this execution.")
```

```

print("\nContents of dataset_manifest_df:")
display(dataset_manifest_df.head())
print("\nColumns in dataset_manifest_df:", dataset_manifest_df.columns.tolist())

# Determine if human_labels_df is needed
# human_labels_df contains original_path, new_path, split, class, age, gender, race
# dataset_manifest_df contains split, label, path (which is new_path)
# human_labels_df contains the specific FairFace demographic labels (age, gender, race)
# which are not in dataset_manifest_df. These labels are valuable for traceability
# and potential future analysis related to the human subset.
# Therefore, human_labels_df provides additional value.

# Determine if fairface_combined_df is needed
# fairface_combined_df contains file, age, gender, race, service_test, path (which is original_path)
# dataset_manifest_df contains split, label, path (which is new_path)
# fairface_combined_df contains the original FairFace labels and original paths for *all* raw FairFace images,
# not just the ones included in the final dataset. It also includes the 'service_test' column.
# This provides traceability back to the original FairFace dataset and its full contents.
# It seems fairface_combined_df was generated in a commented-out cell. Let's assume it was intended.
# It provides the link between original FairFace filenames/paths and their full labels.
# While not strictly necessary for the final ImageFolder, it is useful for full traceability.
# Given the original goal of creating a manifest for traceability, fairface_combined_df is useful.

print("\n# Decision on CSVs:")
print("# human_labels.csv: KEEP. This file contains the specific age, gender, and race labels for the human images included in the")
print("# fairface_combined_labels.csv: KEEP (if generated). This file contains the original FairFace labels and original paths for

```


